



DevOps ACADEMY

BREAKING THE WALLS
BETWEEN DEV AND OPS



—

Microservices & Containers in the Cloud



Expectations & Modus Operandi



RECORDING DISCLAIMER



Agenda

1. SEPARATING THE CONCEPTS (AND BRINGING THEM TOGETHER)
2. DOCKER (DOCKER, DOCKER IN THE CLOUD DEMO, A MENTION TO KNATIVE)
3. KUBERNETES
4. KUBERNETES IN THE CLOUD (GKE IN GCP) – DEMOS
5. Q&A

1

Separating the Concepts



—

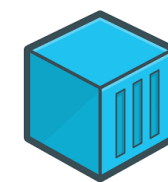
Microservices & Containers in the Cloud



Microservices

[...] an architectural style that structures an application as a collection of services that are:

- Independently deployable
- Loosely coupled
- Organized around business capabilities
- Owned by a small team



Containers

A container is a standard unit of software that packages up code and all its dependencies, so the application runs quickly and reliably from one computing environment to another.



Cloud

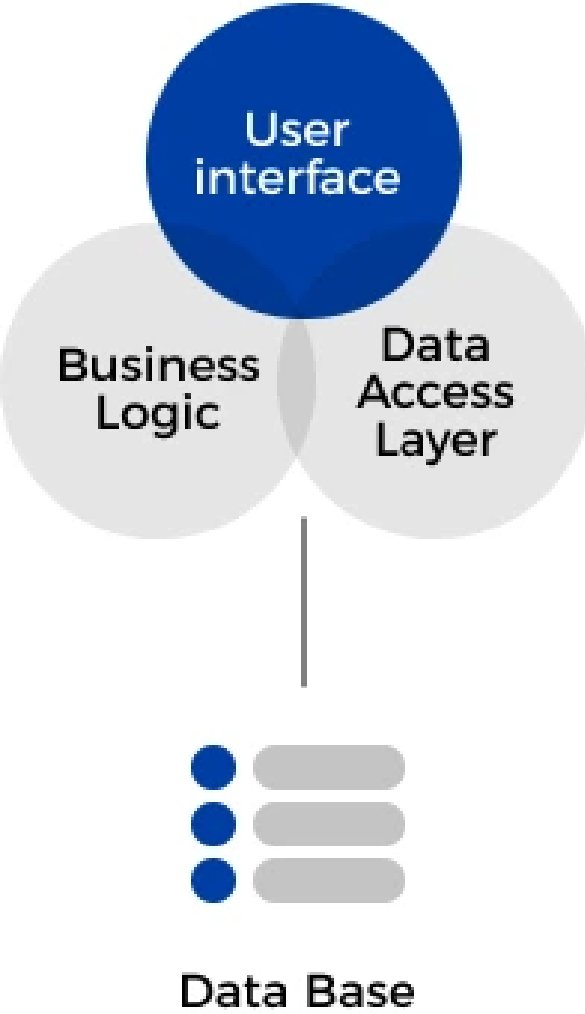
[...] it's a term used to describe a global network of servers, each with a unique function. The cloud is not a physical entity, but instead is a vast network of remote servers around the globe which are hooked together and meant to operate as a single ecosystem.

Microservices

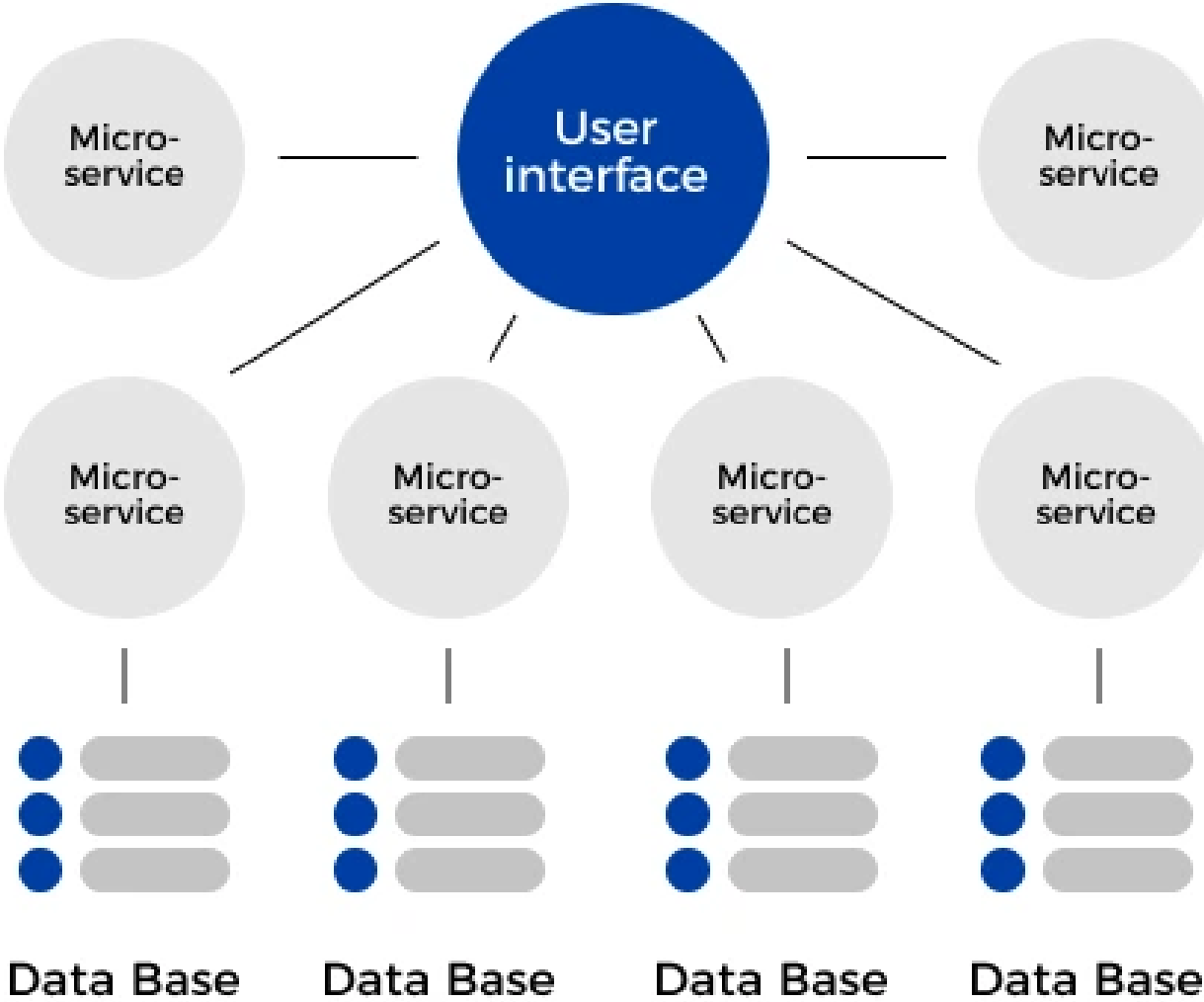
What are  Microservices?

Monolithic vs Microservices

MONOLITHIC ARCHITECTURE



MICROSERVICE ARCHITECTURE

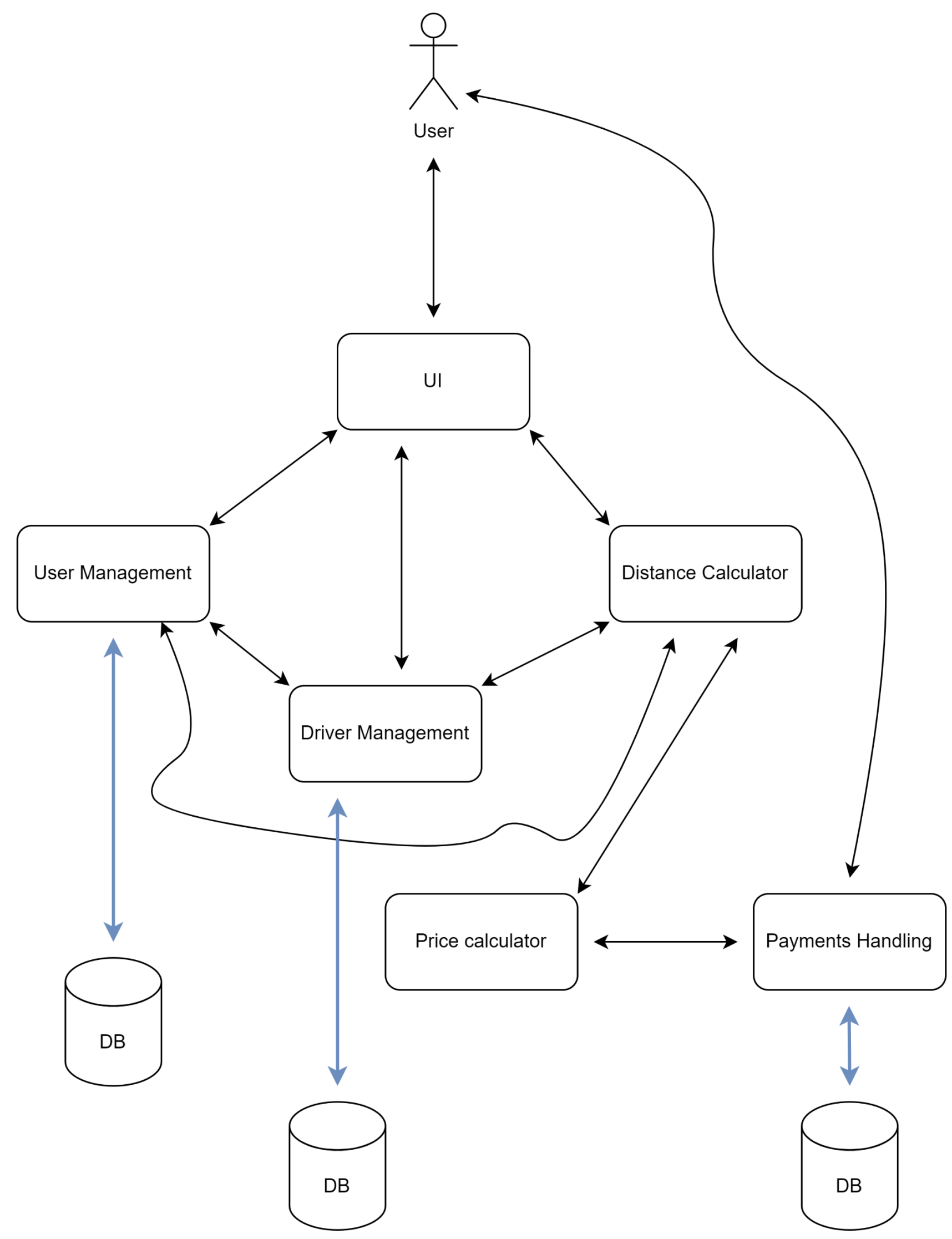
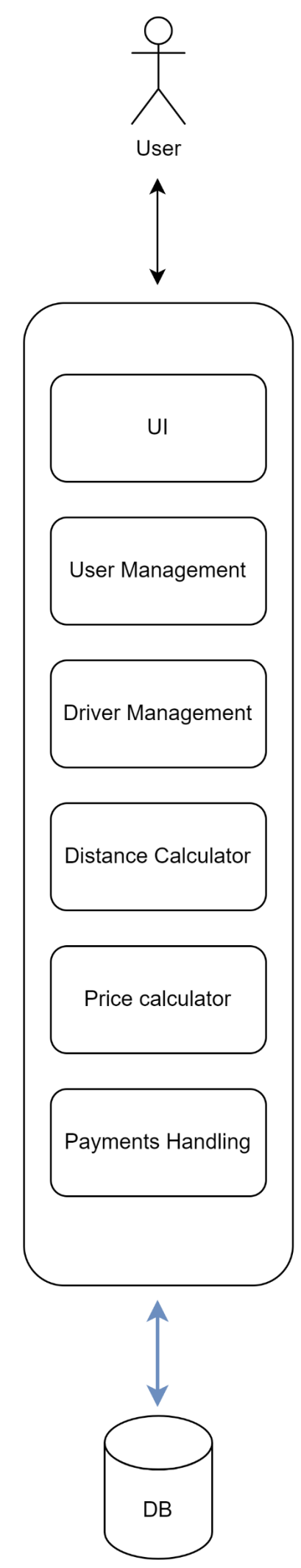


	Microservices	Monoliths
Architecture	Collection of small services	Single build of a unified code
Scalability	Precise scaling and better usage of resources	Hard to scale
Time to Market	Easy to build and deploy	Complicated and time-consuming deployments
Reliability	Very reliable. If service fails, the application will not go down as a whole	If service fails, the entire application goes down





Monolithic example



But wait... is it all bad?

PROS		CONS
Monolithic	Less cross-service concerns. Seamless debugging and testing. Simpler deployment. Easier development.	Code complexity. Highly interdependent components. Limited scalability. New technology barriers.
Microservices	Easier maintenance. Better scalability. More flexibility in choosing the technology. Service isolation.	Extra complexity (in general). System distribution. More cross-service concerns. Harder testing.

Containers (Engine agnostic)

“A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another. A container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.”

Standard: Containers are an industry standard, so they could be *portable* anywhere.

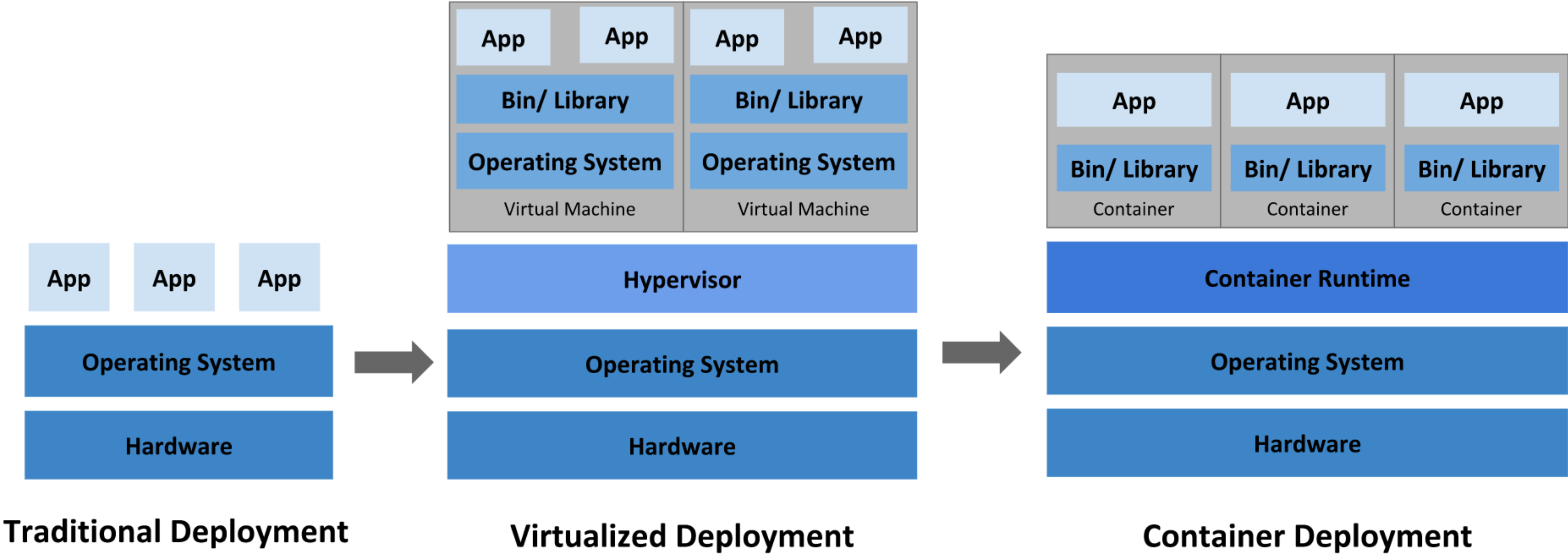
Lightweight: Containers share the machine’s OS system kernel and therefore do not require an OS per application, driving higher server efficiencies and reducing server and licensing costs.

Secure: Given the intrinsic *isolation* capabilities of containers, applications packaged in containers are safer.





Containers (Engine agnostic)



Containers (Engine agnostic)

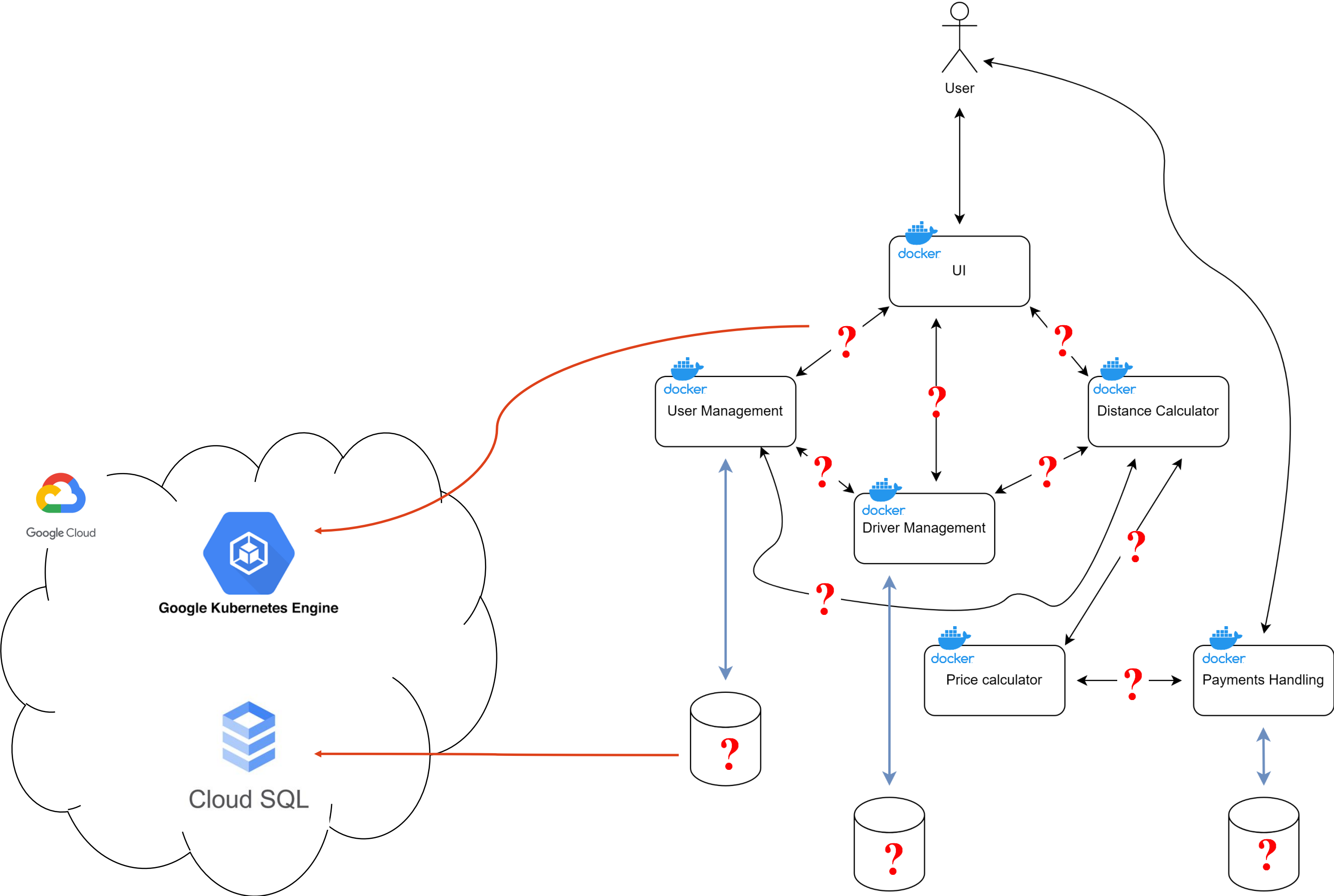


Runtimes (high and low level)

Cloud (more on that later)

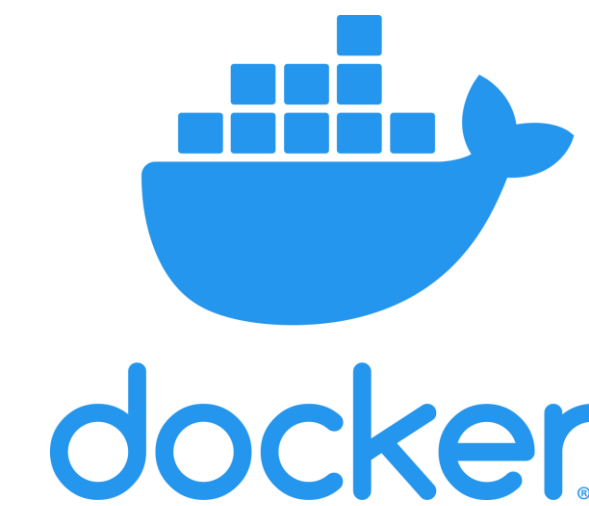


Bringing the concepts together



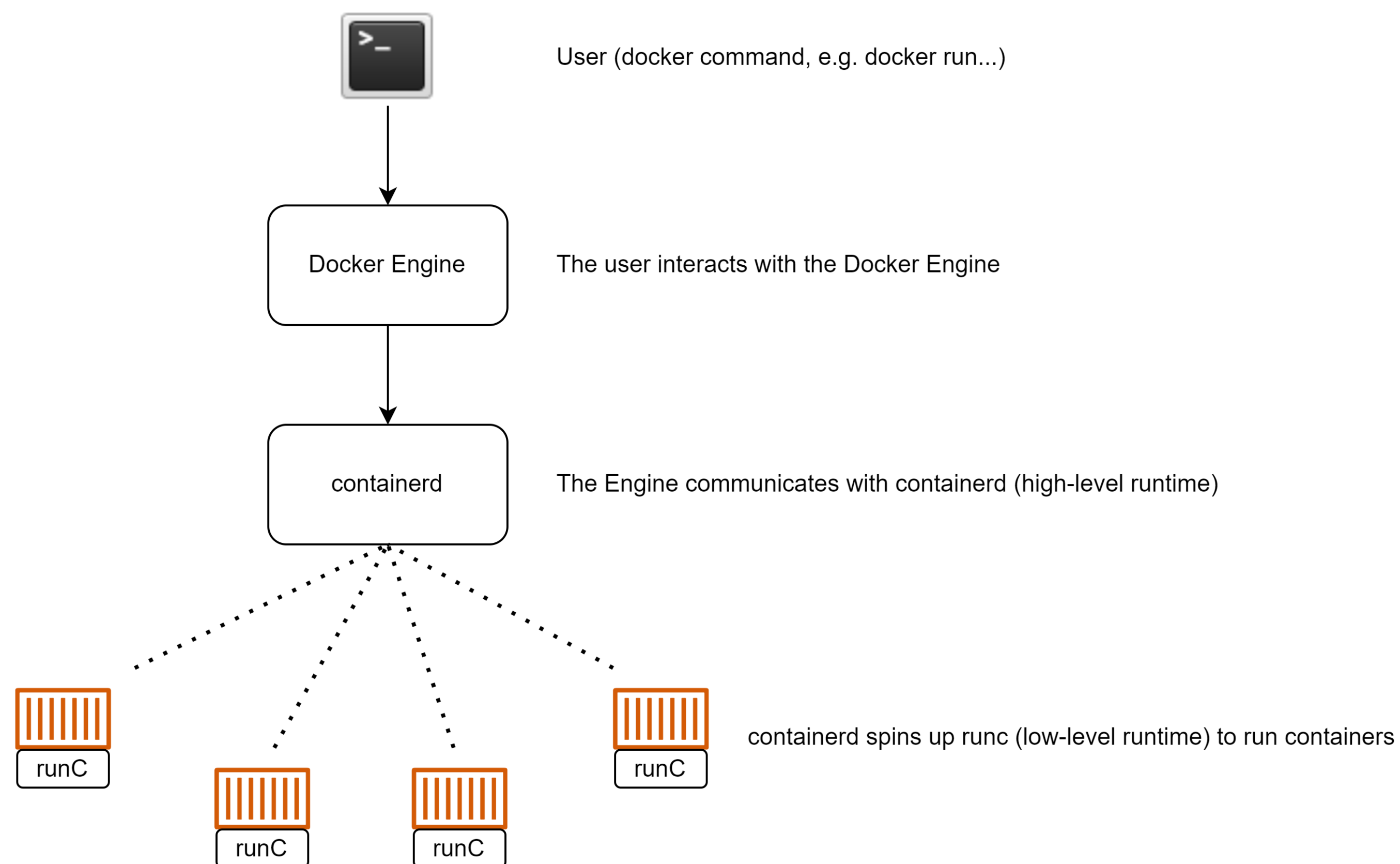
2

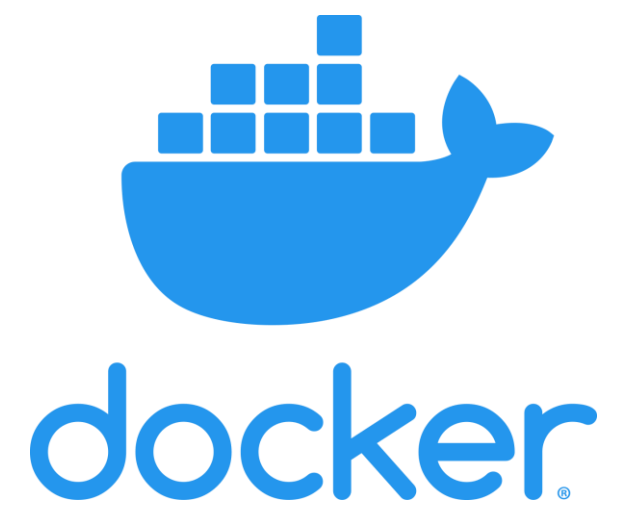
Docker



Docker

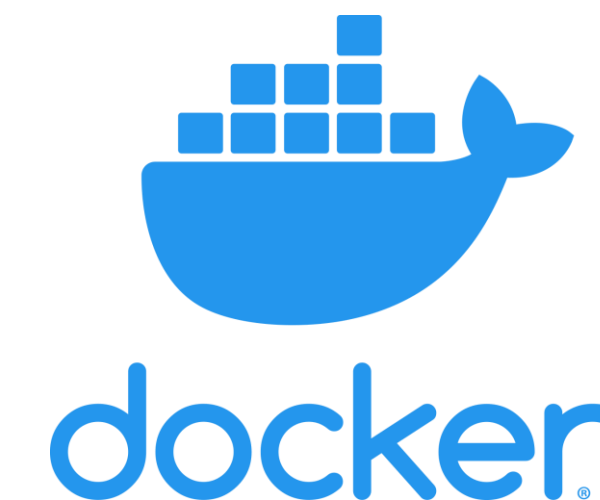
- a container *engine*, not containers per se.
- a facilitator and enabler of application development.
- it still needs a container runtime, such as *containerd*.





Let's demo some Docker... in the cloud!

- we'll spin up a container registry in GCP
- we'll pull an image from the Docker public registry and push it to our private GCP container registry
- we'll also build an image with Docker and push it to our private registry
- we'll finally run our images locally, in plain Docker (no orchestration)



Knative and Cloud Run



Knative (pronounced kay-nay-tiv) is an open source community project which adds components for deploying, running, and managing serverless, cloud-native applications. Knative eliminates the tasks of provisioning and managing servers.



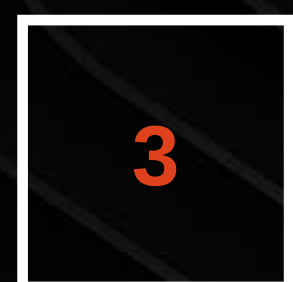
Google Cloud Run

Google Cloud Run is fully-managed Knative-based serverless platform. You manage no infrastructure, so Cloud Run lets you go from container to production in seconds. Cloud Run is a managed compute platform that enables you to run containers that are invocable via requests or events. Cloud Run is serverless: it abstracts away all infrastructure management



https://endava-my.sharepoint.com/personal/alina_alexandru_endava_com/_layouts/15/stream.aspx?id=/personal/alina_alexandru_endava_com/Documents/Recordings/Introduction+To+Computing+In+Google+Cloud+_+DevOps+Academy-20221125_160200-Meeting+Recording.mp4&sw=auth





Kubernetes



Kubernetes

“It’s an open source container orchestration system”

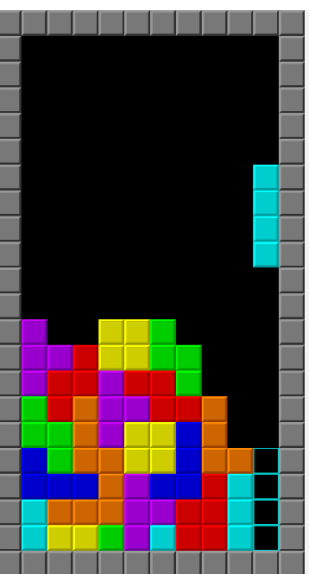
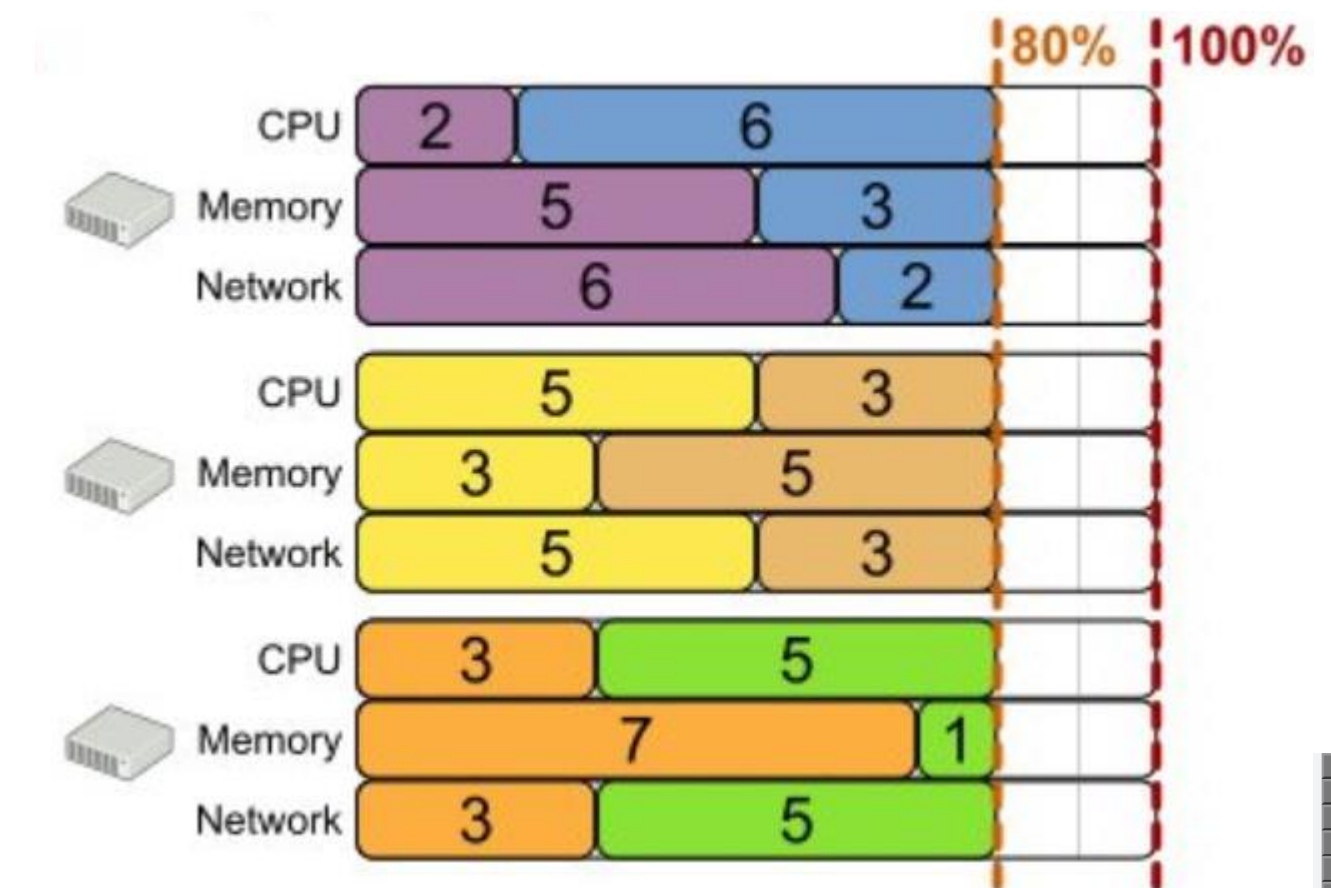
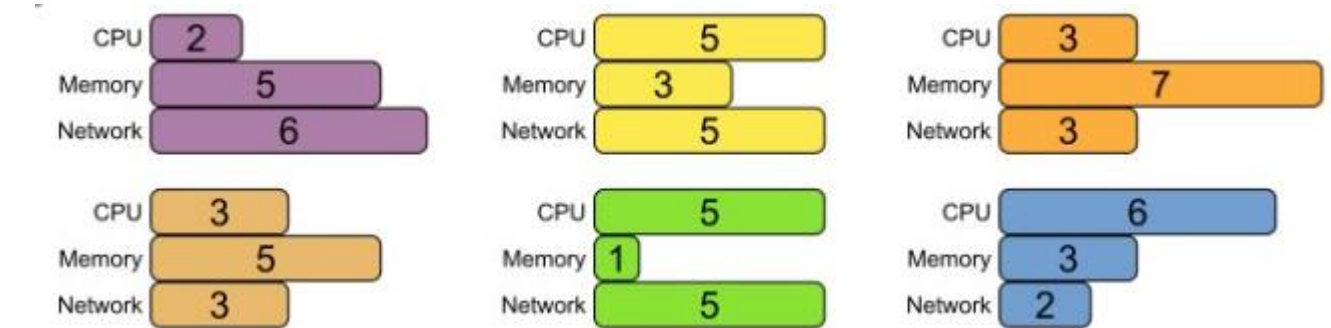
- Portable
- Extensible
- Containerized workloads
- Declarative configuration
- Automation
- Open source





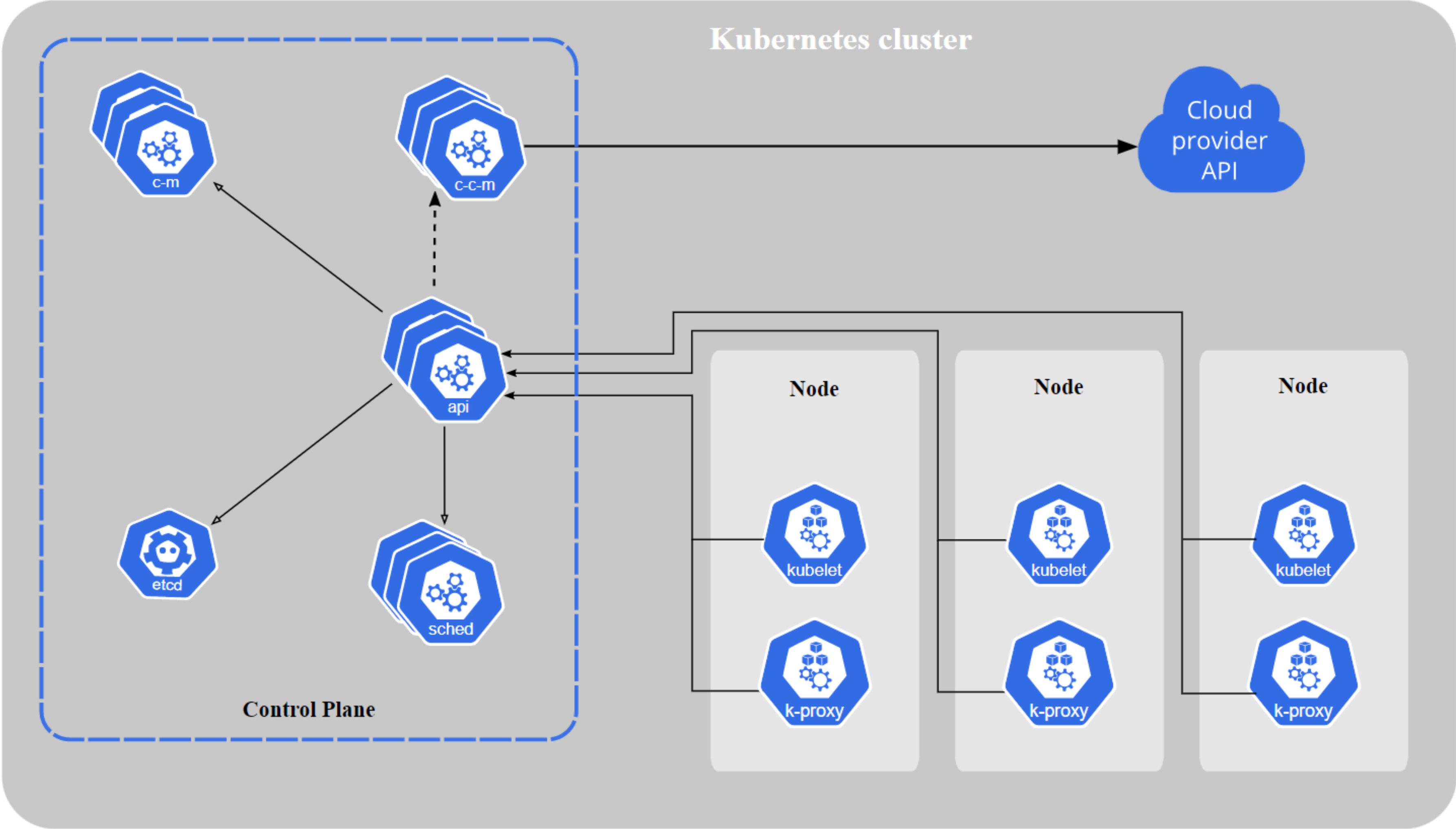
Kubernetes (but why?)

- Service discovery and load balancing
- Storage orchestration
- Automated rollouts and rollbacks
- Automatic bin packing
- Self-healing
- Secret and configuration management



<https://youtu.be/Vt1iS5q1uzk?t=658>

Kubernetes Components



Kubernetes Components (Control Plane)



API SERVER

The API server is a component of the Kubernetes control plane that exposes the Kubernetes API. The API server is the front end for the Kubernetes control plane.



CONTROLLER MANAGER

Control plane component that runs controller processes. Logically, each controller is a separate process, but to reduce complexity, they are all compiled into a single binary and run in a single process.



ETCD

Consistent and highly-available key value store used as Kubernetes' backing store for all cluster data.



SCHEDULER

Control plane component that watches for newly created Pods with no assigned node, and selects a node for them to run on.



CLOUD CONTROLLER MANAGER

A Kubernetes control plane component that embeds cloud-specific control logic. The cloud controller manager lets you link your cluster into your cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with your cluster. The cloud-controller-manager only runs controllers that are specific to your cloud provider.

Kubernetes Components (Nodes)



KUBELET

An agent that runs on each node in the cluster. It makes sure that containers are running in a Pod.



KUBE-PROXY

kube-proxy is a network proxy that runs on each node in your cluster, implementing part of the Kubernetes Service concept.

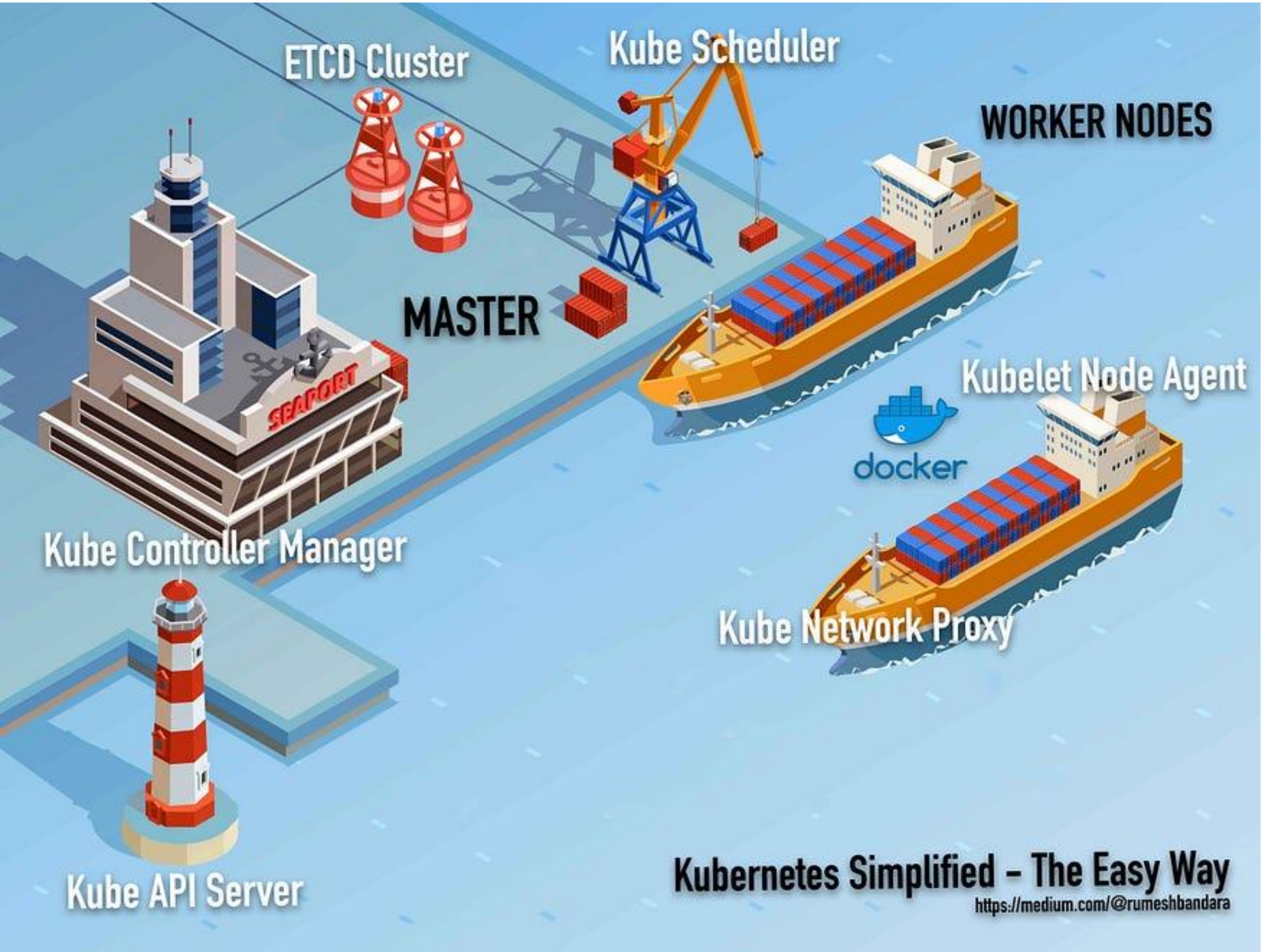
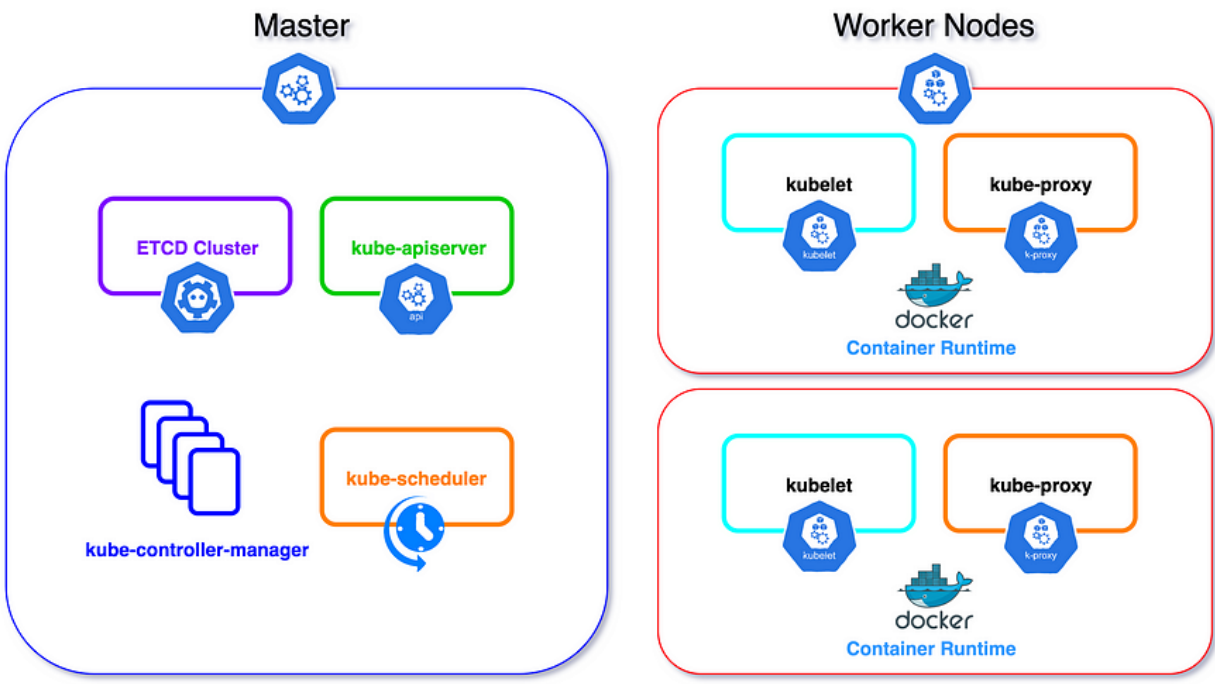


CONTAINER RUNTIME

The container runtime is the software that is responsible for running containers.

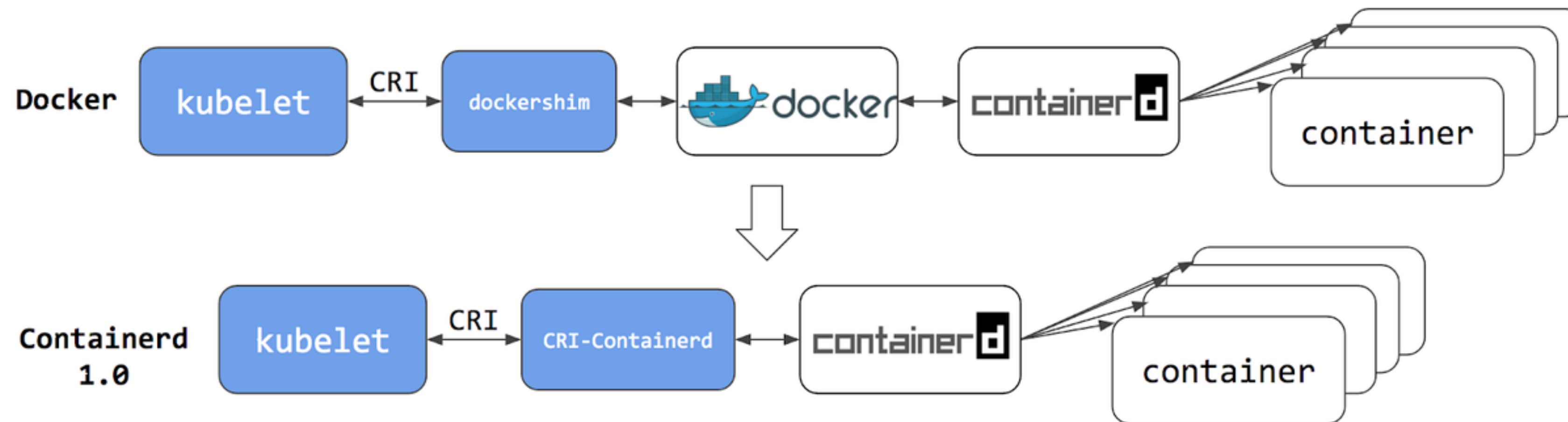
Kubernetes Components

(Cofused???)



Kubernetes Simplified – The Easy Way
<https://medium.com/@rumeshbandara>

But whait... I heard in tech news that Kubernetes id dropping Docker support... what up?



Some Kubernetes components

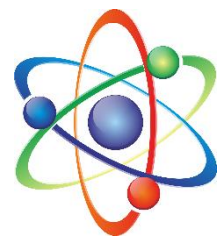
- POD 
- Replica Set 
- Deployment 
- Daemon Set 
- Stateful Set (PV and PVC ↔ GCP disk) 
- Jobs & CronJobs  
- Config Map 
- Service 
- Ingress (GCP Load Balancer) 
- Namespaces
- More...

POD

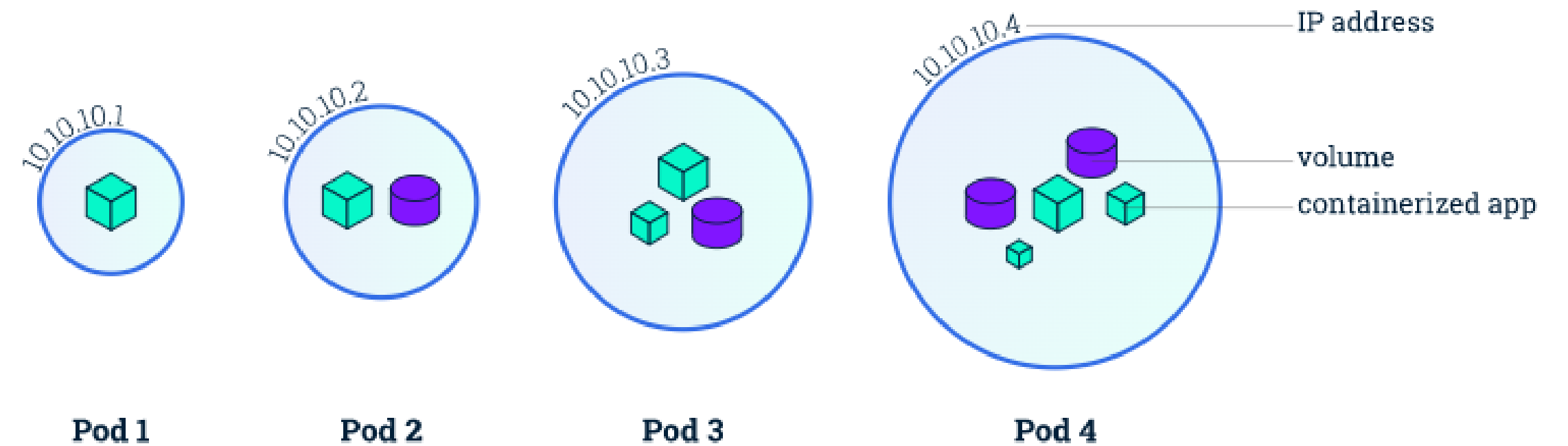
“Pods are the smallest deployable units of computing that you can create and manage in Kubernetes.”



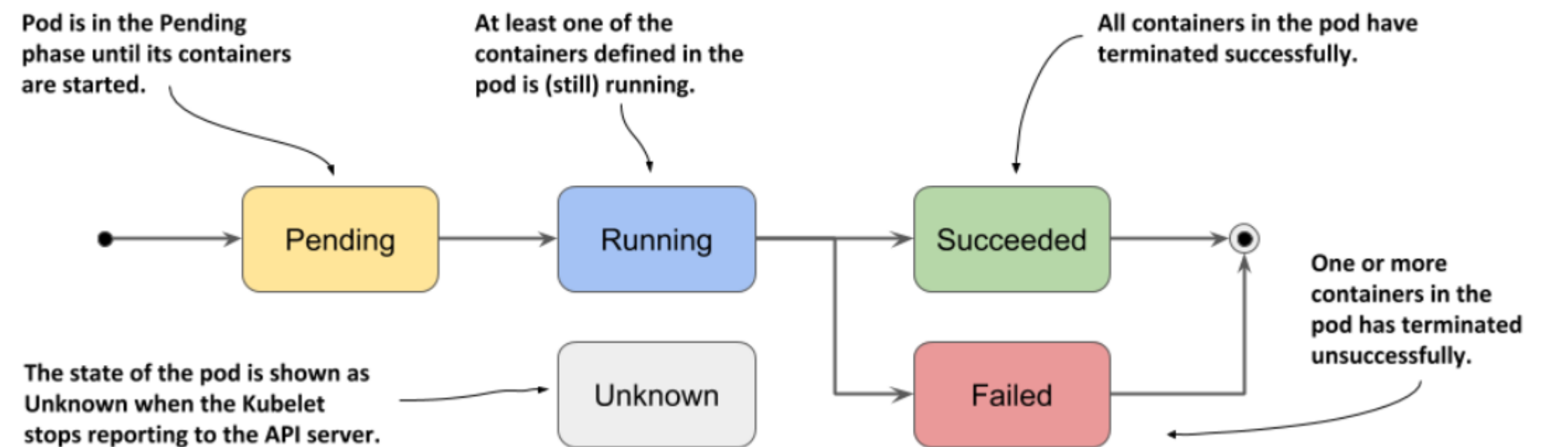
- Smallest execution unit in Kubernetes
- Encapsulates one or more containers
- Ephemeral
- It gets it's own IP



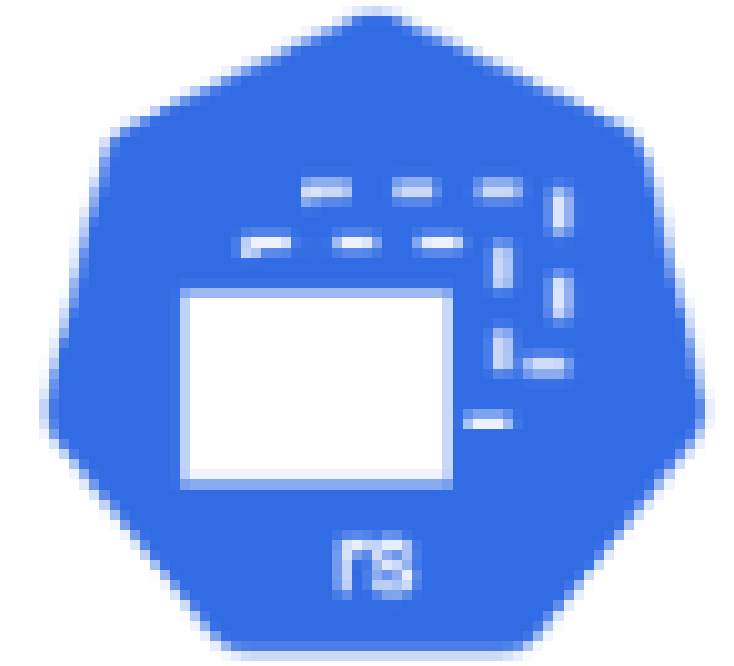
```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```



Lifecycle of PODS:



Replica Set



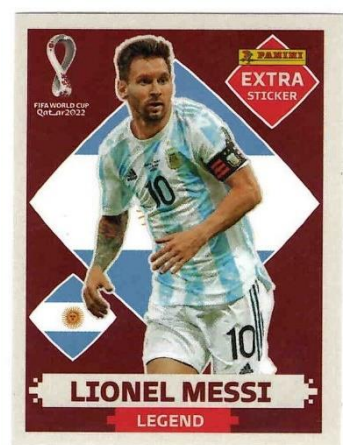
“A ReplicaSet's purpose is to maintain a stable set of replica Pods running at any given time. As such, it is often used to guarantee the availability of a specified number of identical Pods.”

- Identifies the PODs with Selectors
- Creates or deletes pods as needed to reach the desired number of replicas
- It's linked to the PODs via the POD's metadata.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  # modify replicas according to your case
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: php-redis
          image: gcr.io/google_samples/gb-frontend:v3
```

POD

Selectos and Labels



Foot: Left
Country: Argentina
Club: PSG
Continent: South America
Ligue: France



Foot: Both
Country: Belgium
Club: Manchester City
Continent: Europe
Ligue: England



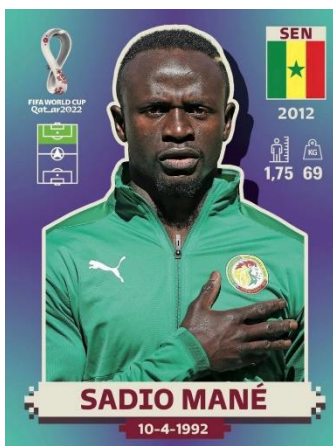
Foot: Right
Country: France
Club: Real Madrid
Continent: Europe
Ligue: Spain



Foot: Left
Country: Sweden
Club: AC Milan
Continent: Europe
Ligue: Italy



Foot: Both
Country: Poland
Club: Barcelona
Continent: Europe
Ligue: Spain



Foot: Both
Country: Senegal
Club: Bayern Munich
Continent: Africa
Ligue: Germany

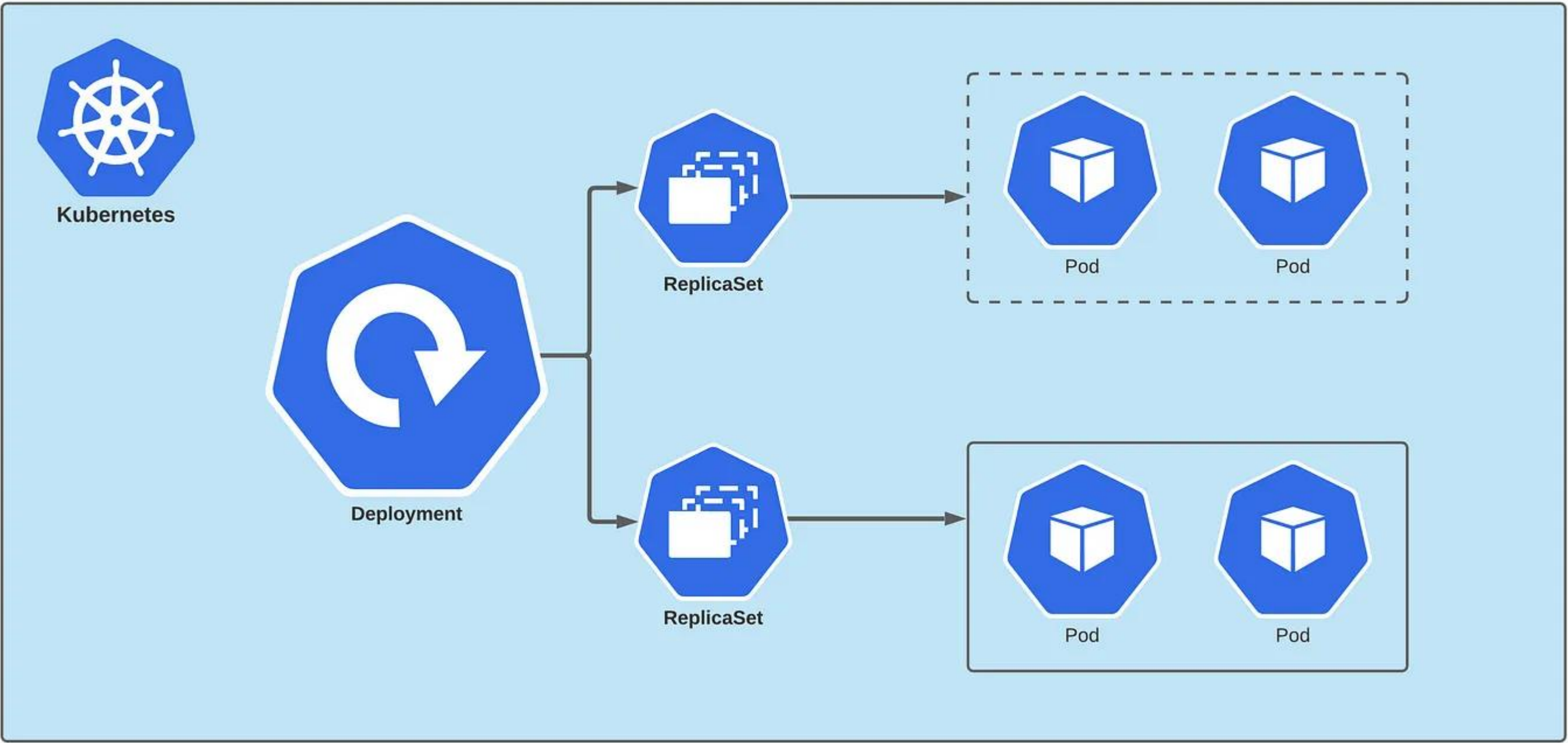


Deployments

"A Deployment provides declarative updates for Pods and ReplicaSets."

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

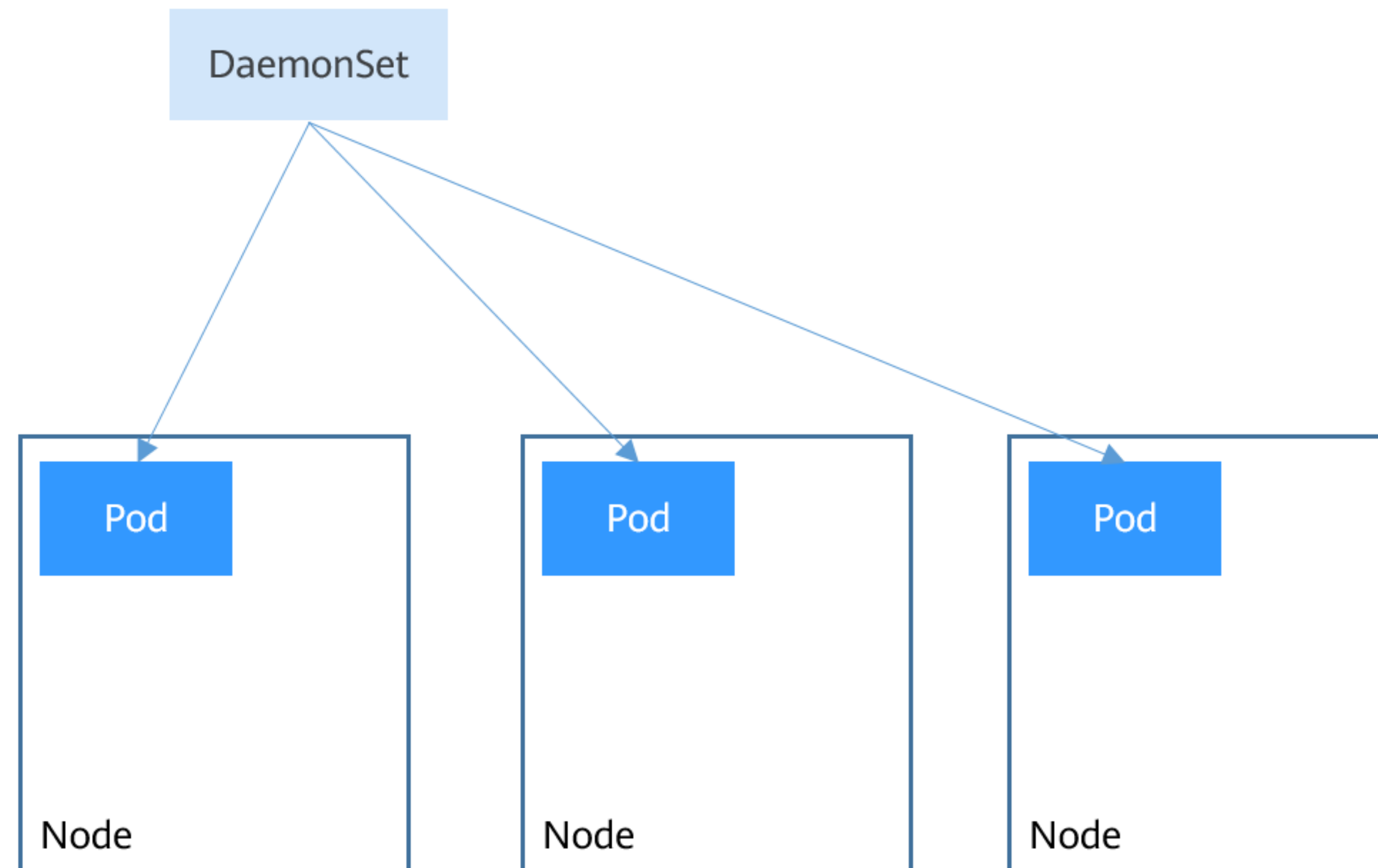
POD



Daemon Sets



"A DaemonSet ensures that all (or some) Nodes run a copy of a Pod."

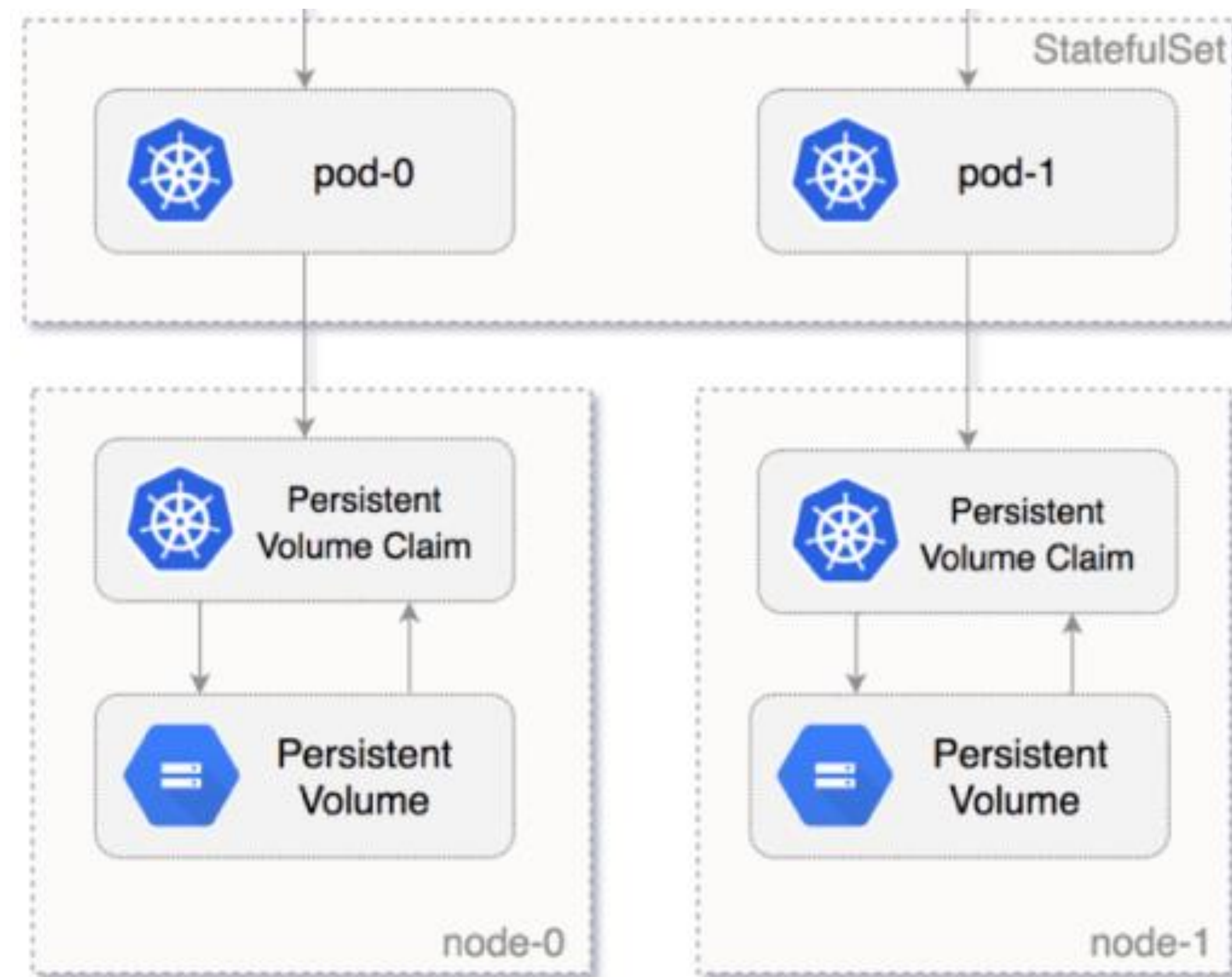




Stateful Sets

"StatefulSet is the workload API object used to manage stateful applications."

- It guarantees uniqueness of the POD.
- Maintain sticky identities.
- PODs are not interchangeable.
- Allows for usage of storage volumes.

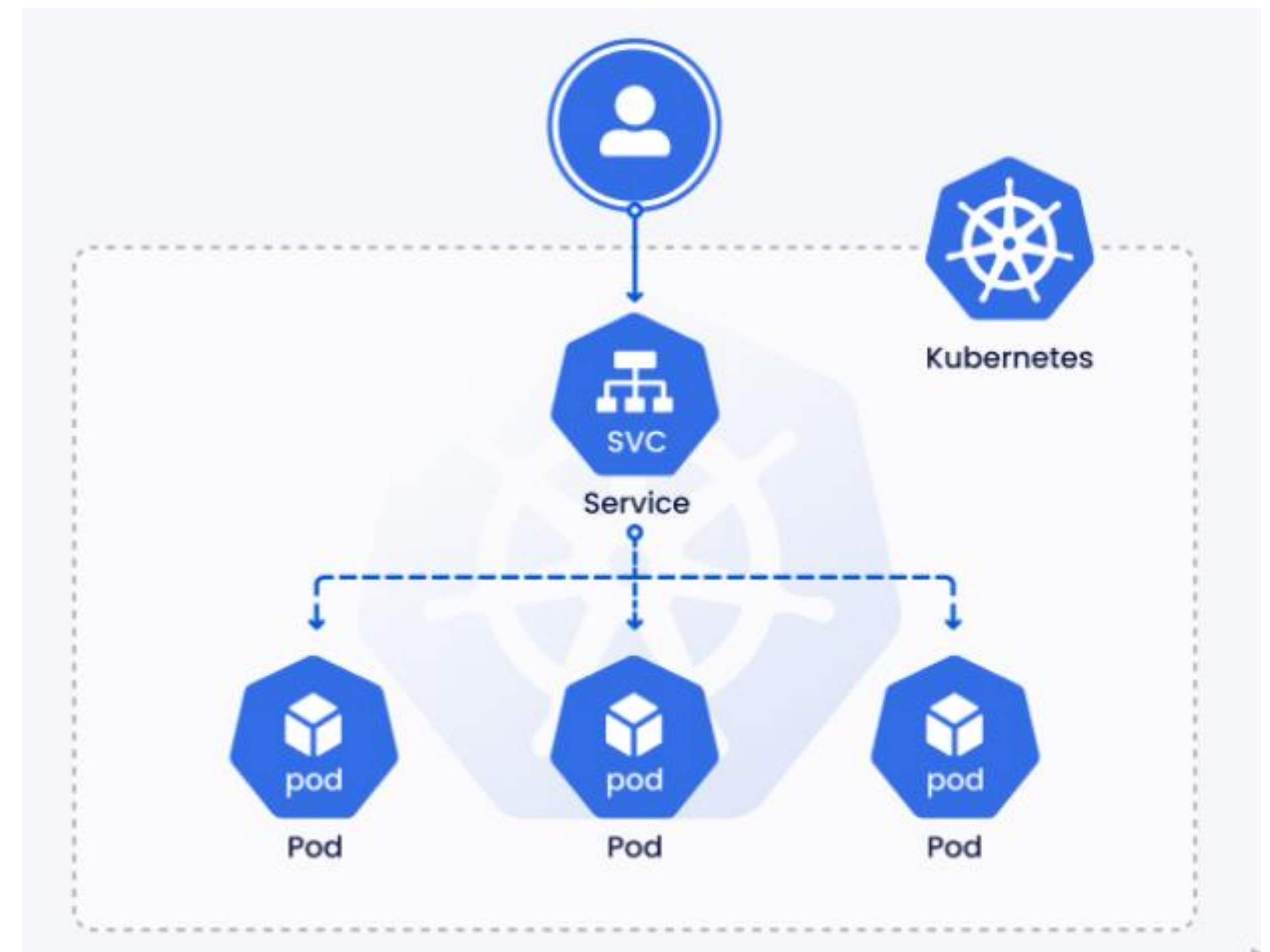


Service



"[...] is an abstraction to help you expose groups of Pods over a network."

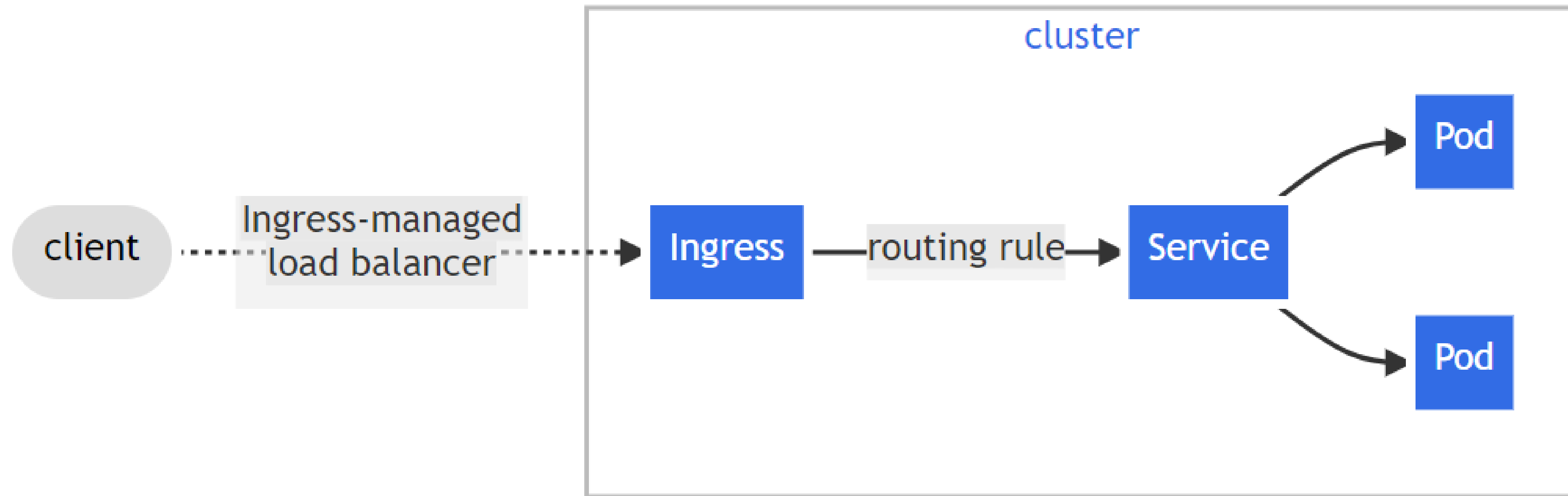
```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```



Ingress



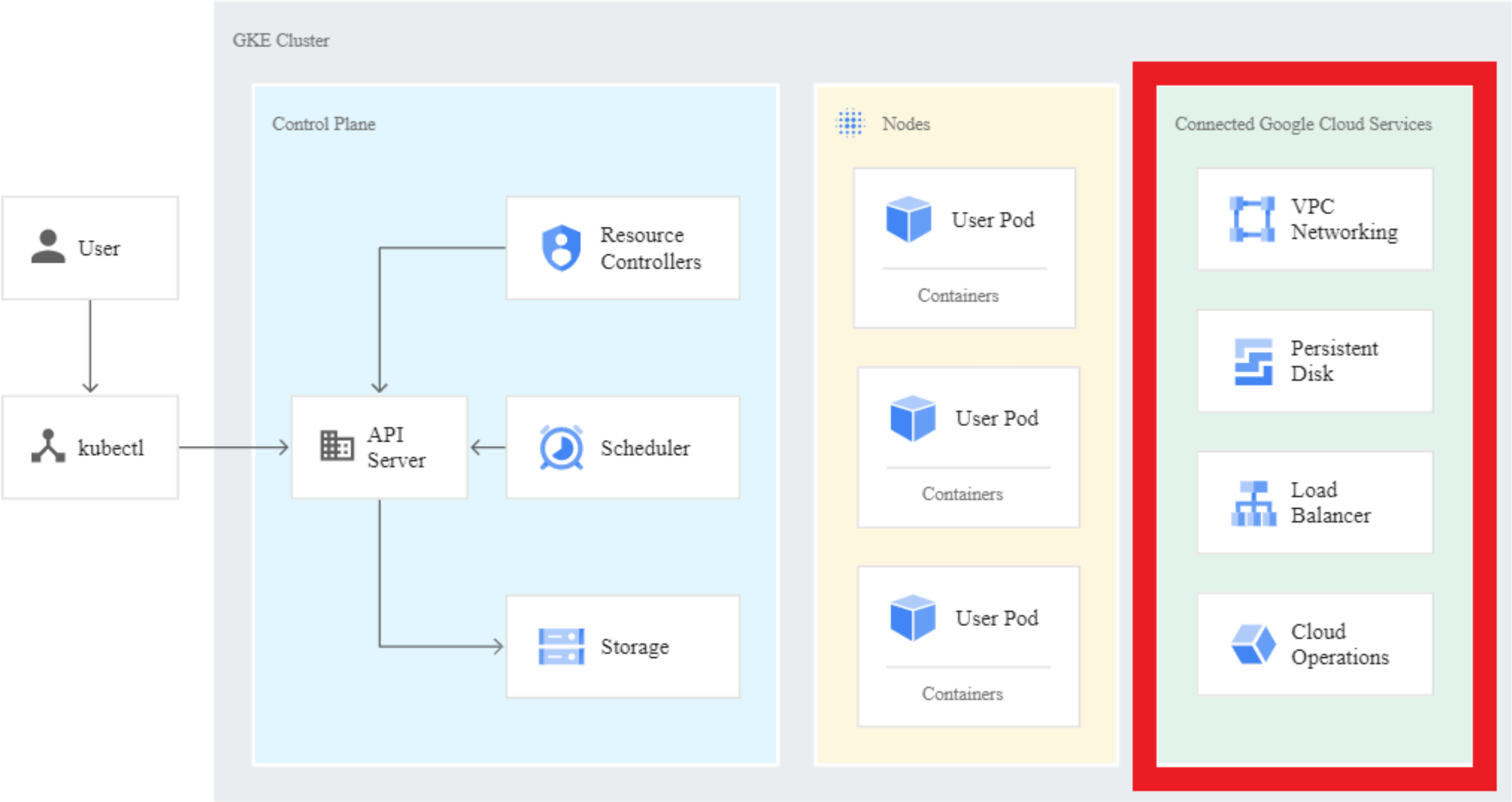
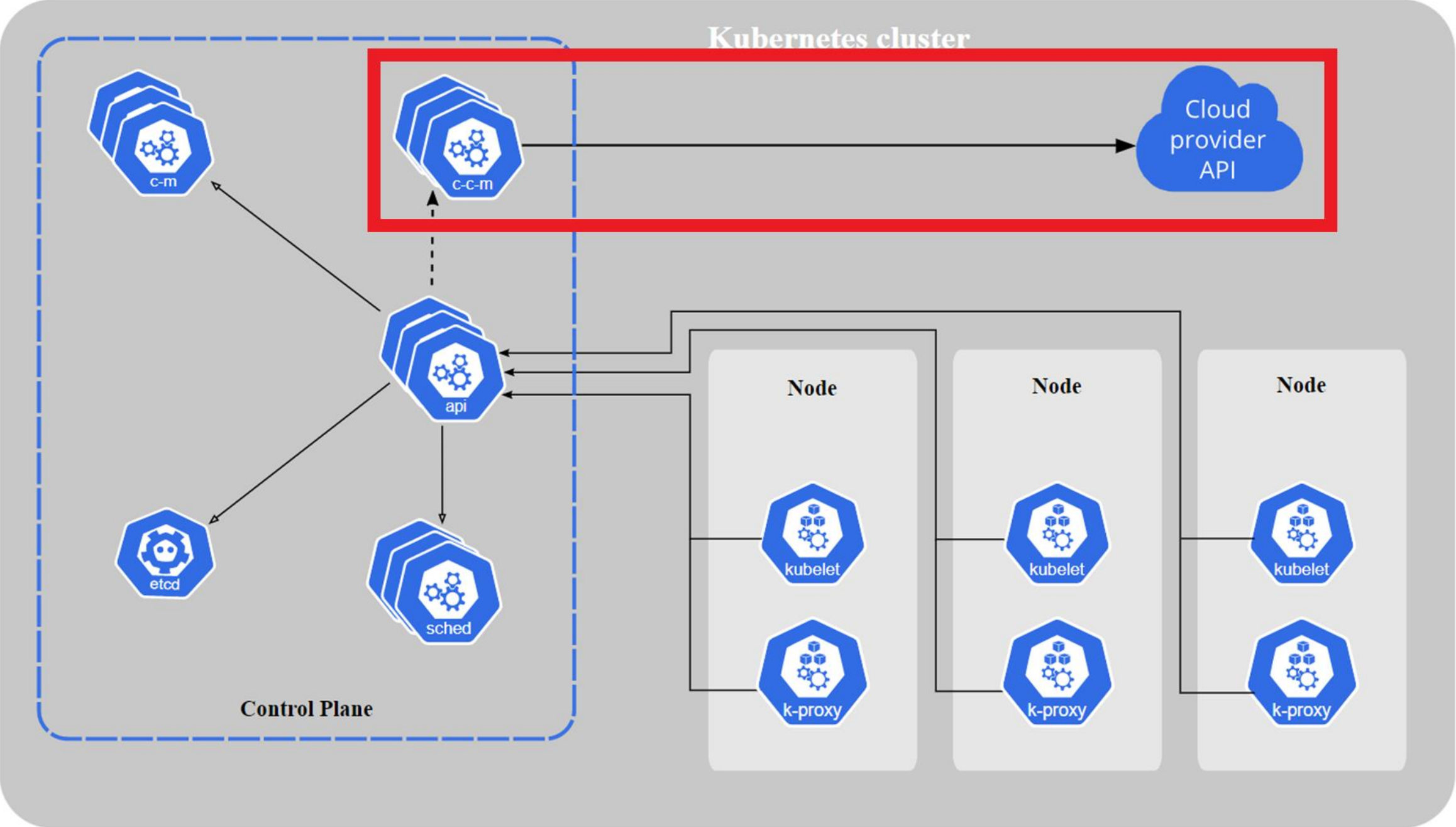
"An API object that manages external access to the services in a cluster, typically HTTP."



4

Kubernetes in the Cloud (GKE in GCP)

Kubernetes and GCP



Kubernetes and GCP

- Let's create a K8s cluster together in the cloud... using the UI.
- Let's demo PODs, ReplicaSet, Stateful Set (with PV and PVC), Deployments, Daemonsets
- Let's also demo Service and Ingress.
- Let's aaaaaalso play with labels and selectors (if there's time).

5

Q&A

